

Compile-Fused Inference Engine (CFIE)

Compiler-Native Inference Optimization Through Persistent Kernel Fusion, Static KV Planning, and Compiled Speculative Decoding in NeuralScript

Authors: Brandon Wiemer

Date: April 2026

Status: Research Proposal

Abstract

We propose **CFIE (Compile-Fused Inference Engine)**, a suite of six compiler-native inference optimizations that exploit NSL's ahead-of-time compilation, cost model, and PTX synthesis to eliminate runtime overhead that vLLM, TensorRT-LLM, and SGLang cannot avoid. The key insight: inference serving frameworks spend significant time on decisions that are *knowable at compile time* — KV cache block sizes, speculative tree structures, sampling strategies, grammar FSM transitions, and decode-phase kernel configurations — but must resolve them at runtime because they operate on pre-compiled kernels with runtime dispatch. NSL generates inference binaries where these decisions are baked into the kernel control flow, the memory layout, and the PTX instructions themselves.

1. The Runtime Tax in LLM Inference

Modern LLM inference engines (vLLM, TensorRT-LLM, SGLang, FlashInfer) achieve high throughput through runtime systems: continuous batching schedulers, paged KV-cache managers, dynamic kernel dispatch. But each runtime decision carries overhead:

Runtime Decision	Overhead	Frequency
Scheduler iteration (batch assembly)	~50-200μs	Every decode step
PagedAttention block table lookup	~5-15μs per request	Every attention layer
Kernel launch (separate per op)	~3-10μs per launch	~15-30 launches per layer
Sampling (separate kernel)	~10-30μs	Every decode step
Grammar FSM state transition	~5-20μs	Every decode step
Speculative tree management	~20-100μs	Every speculative step

For a model with 32 layers and batch=1 (latency-sensitive), the per-token overhead budget is ~100-500µs. The runtime tax above can consume **30-60% of this budget**, leaving only 40-70% for actual matrix computation.

NSL eliminates this tax by making these decisions at compile time.

2. Six Compiler-Native Inference Optimizations

2.1 Optimization 1: Compile-Time KV Cache Layout Planning

Problem: vLLM's PagedAttention allocates KV cache in variable-sized blocks at runtime. Each attention kernel must perform a block table lookup to find the physical location of each logical KV position. This indirection adds HBM reads and control flow to every attention computation.

NSL's approach: The compiler knows the model's architecture (n_kv_heads, head_dim, n_layers) and the target GPU's HBM capacity at compile time. It plans the KV cache layout statically:

```
Compile-time KV plan for NSLCoder-50M on RTX 5070 Ti (16GB):
  Model weights: 97.6 MB (BF16)
  Available for KV: 16384 - 98 - 800 (CUDA) = 15,486 MB

  Per-token KV: 4 KV heads × 128 head_dim × 2 (K+V) × 8 layers × 2 bytes = 16,384
bytes
  Max tokens: 15,486 MB / 16 KB = 968,000 tokens

  Layout: contiguous [n_layers][2][max_tokens][n_kv_heads][head_dim]
          (layers × K/V × sequence × heads × dim)

  Block size: 256 tokens (aligned to GPU cache line and SMEM tile size)
  Blocks per sequence: ceil(max_seq / 256) = 8 blocks for seq=2048
```

The compiled attention kernel uses **direct indexing** into this pre-planned layout — no block table, no indirection, no runtime allocation. New requests get a contiguous slice of the pre-allocated buffer. The `(serve)` block's scheduler simply maintains a free-list of sequence slots, not a complex block allocator.

Integration with M25 (PagedAttention): NSL's existing `(block_alloc.rs)` and `(page_table.rs)` are retained for the general case (dynamic sequence lengths, preemption). But when the compiler determines the workload is predictable (e.g., fixed max_seq_len, known batch size), it generates the static layout variant. The `(serve)` block config controls this:

```
ns!
```

```
serve(model=m, port=8080):  
    max_seq = 2048  
    max_batch = 64  
    kv_layout = static    # compiler plans layout; omit for dynamic PagedAttention  
    ...
```

2.2 Optimization 2: Fused Decode-Sample Kernel

Problem: The decode phase of LLM inference runs these operations as separate kernels:

```
1. RMSNorm(hidden)           → kernel launch  
2. LM_head matmul(hidden, W) → kernel launch (outputs [batch, vocab] logits)  
3. Softmax(logits / T)       → kernel launch  
4. Top-k filtering           → kernel launch  
5. Top-p (nucleus) filtering → kernel launch  
6. Multinomial sampling      → kernel launch
```

Six kernel launches for the final token selection. The logits tensor [batch, vocab_size] is written to HBM after the matmul, read back for softmax, written again, read for top-k, etc. For vocab=49152 and batch=1: $49152 \times 4 \text{ bytes} \times 6 \text{ HBM round-trips} = \mathbf{1.1 \text{ MB of wasted HBM traffic}}$ per token.

NSL's approach: The compiler generates a **single fused decode-sample kernel** that never materializes the full logits tensor in HBM:

Fused Decode-Sample Kernel (one kernel launch):

Input: hidden_state [1, d_model], LM_head weights [d_model, vocab]

Output: sampled_token_id (single int)

Algorithm (tiled, on-chip):

```
// Phase 1: Tiled matmul + running top-k in SMEM
running_topk = MinHeap(k=50) // in registers
running_max = -inf

for vocab_tile in 0..ceil(vocab/TILE):
    logits_tile = hidden @ W_tile           // [1, TILE] - in SMEM
    logits_tile = logits_tile / temperature // fused with matmul epilogue

    for i in 0..TILE:
        if logits_tile[i] > running_topk.min():
            running_topk.replace_min(logits_tile[i], vocab_tile*TILE + i)
        running_max = max(running_max, logits_tile[i])

// Phase 2: Softmax over top-k candidates only (k=50, not vocab=49152)
sum_exp = 0
for (logit, idx) in running_topk:
    prob = exp(logit - running_max)
    sum_exp += prob

// Phase 3: Top-p filtering + multinomial sampling
sorted_probs = sort(running_topk by prob desc)
cumsum = 0
for (prob, idx) in sorted_probs:
    cumsum += prob / sum_exp
    if cumsum > top_p:
        break
    candidates.push((prob, idx))

// Phase 4: Sample from candidates
r = random_uniform()
sampled_token = weighted_sample(candidates, r)

// Write ONLY the token ID to HBM - logits never leave the chip
output[0] = sampled_token
```

HBM savings: The full logits tensor ($[1, 49152] \times 4$ bytes = 192KB) is never written to HBM. Only the final token ID (4 bytes) exits the kernel. **48,000× reduction in output HBM traffic.**

FLOP savings: Softmax is computed over k=50 candidates, not vocab=49152. **~1000× fewer exp() calls.**

Integration with M44 (Grammar Compiler): When grammar-constrained decoding is active, the compiled DFA's state transitions are fused into Phase 1. The kernel checks each vocab tile against the DFA's valid-token set for the current state, skipping entries that would violate the grammar:

```
for i in 0..TILE:
    token_id = vocab_tile*TILE + i
    if not grammar_dfa.is_valid(current_state, token_id):
        continue // skip - never enters top-k
    if logits_tile[i] > running_topk.min():
        running_topk.replace_min(logits_tile[i], token_id)
```

The DFA transition table is embedded as a compile-time constant in the kernel — no runtime FSM interpreter.

2.3 Optimization 3: Compiled Speculative Decoding

Problem: Speculative decoding (M33) runs a draft model K times autoregressively, then verifies all K tokens with one target model forward pass. The draft tree structure, verification logic, and rejection sampling are managed at runtime via `DraftModelRunner` and `verify.rs`.

NSL's approach: The compiler knows both the draft and target models at compile time. It generates a **single compiled speculative step** that bakes the draft tree structure, token count K, and verification logic into the kernel control flow:

```
ns!

serve(model=target, port=8080):
    speculative:
        draft = DraftModel.load("draft_50m.nslm")
        tokens = 5           # K=5 draft tokens
        method = standard    # or medusa, lookahead
        temperature = 0.0    # greedy draft
```

The compiler generates:

Compiled Speculative Step:

```
1. Draft phase (5 iterations, fused into persistent kernel):
    for k in 0..5:
        draft_logits = draft_forward(draft_kv, seq_pos + k) // compiled draft
model
        draft_token[k] = argmax(draft_logits) // fused sampling
        draft_probs[k] = softmax(draft_logits)[draft_token[k]] // for rejection
sampling

2. Target verification (single forward, batch=5+1):
    target_logits = target_forward(tokens=[input + draft_tokens], batch_pos=5+1)
    // CSHA-fused attention over all 5+1 positions

3. Rejection sampling (fused into target kernel epilogue):
    accepted = 0
    for k in 0..5:
        target_prob = softmax(target_logits[k])[draft_token[k]]
        if random() < target_prob / draft_probs[k]:
            accepted += 1
        else:
            // Resample from adjusted distribution
            correction_token = sample(target_logits[k] - draft_probs)
            break

4. KV cache update:
    // Advance KV cache by `accepted` positions
    // Discard draft KV entries for rejected tokens
```

Key advantage: The draft model is a compiled NSL function, not a separate Python model. Both draft and target share the same binary, the same CUDA context, and the same memory pool. The draft model's CSHA-fused kernels are generated alongside the target's — no separate inference engine needed.

Tree attention variant: When `(method = tree)`, the compiler generates a tree-structured draft where the top-2 tokens at each position are explored (tree width=2, depth=K). The tree attention mask is a **compile-time constant** embedded in the FlashAttention kernel — not a runtime-constructed mask tensor.

2.4 Optimization 4: Persistent Decode Kernel

Problem: Each decode step launches 15-30 separate kernels (norms, projections, attention, FFN, residuals) per layer $\times$ 32 layers = $\sim$ 500-1000 kernel launches per token. At $\sim 5\mu$ s per launch, this is 2.5-5ms of pure launch overhead.

NSL's approach: CSHA Level 3 (full block fusion) applied to the decode phase. For single-token decode with $d_{\text{model}} \leq 2048$, the entire transformer block can be a persistent kernel:

Persistent Decode Block Kernel (1 CTA, 1 kernel launch per layer):

Input: x [1, d_{model}] in global memory

Output: x' [1, d_{model}] in global memory

SMEM allocation: enough for Q/K/V tiles + FFN intermediates

1. Load x into SMEM
2. RMSNorm in SMEM
3. $Q = W_q @ x_{\text{normed}}$ (in SMEM, never touches HBM)
4. $K = W_k @ x_{\text{normed}}$, $V = W_v @ x_{\text{normed}}$
5. RoPE rotation (in registers)
6. Append K,V to KV cache (one HBM write)
7. FlashAttention decode over KV cache (HBM reads for KV cache)
8. $O = W_o @ \text{attn_out}$ (in SMEM)
9. Residual add in SMEM
10. RMSNorm in SMEM
11. $\text{gate} = W_{\text{gate}} @ x_{\text{normed}}$, $\text{up} = W_{\text{up}} @ x_{\text{normed}}$ (in SMEM)
12. $\text{FFN} = W_{\text{down}} @ (\text{silu}(\text{gate}) * \text{up})$ (in SMEM)
13. Residual add
14. Store x' to HBM

Total HBM traffic: read x + read weights + read/write KV cache + write x'

Eliminated: ALL intermediate tensor HBM round-trips

For a 32-layer model: **32 kernel launches** instead of ~1000. At 5 μs per launch: **160 μs saved per token**.

The compiler determines feasibility via SMEM analysis. For $d_{\text{model}}=4096$ (7B models), Level 3 requires ~128KB SMEM — feasible on H100 (228KB per SM) but not on older GPUs. The cost model selects the optimal fusion level per target.

2.5 Optimization 5: Weight-Aware KV Quantization

Problem: KV cache quantization (M42) uses a uniform precision across all layers (e.g., INT8 everywhere). But attention heads in different layers have different sensitivity — early layers often need higher precision for position encoding, while middle layers tolerate aggressive quantization.

NSL's approach: When weights are available (`(ns1 build --weights)`), the compiler performs spectral analysis on attention projection weights to determine per-layer KV precision:

Per-layer KV precision selection:

Layer 0: FP16 K, FP16 V (high position sensitivity – first RoPE layer)

Layer 1: FP16 K, INT8 V (K needs position precision, V is tolerant)

Layers 2-6: INT8 K, INT8 V (middle layers – low sensitivity)

Layer 7: FP16 K, FP16 V (final layer – high impact on output)

The compiled attention kernels are generated with **per-layer dequantization logic** embedded in the kernel. Layer 3's attention kernel reads INT8 K/V and dequantizes in registers; layer 0's kernel reads FP16 directly. No runtime precision dispatch.

KV memory savings (NSLCoder-50M, seq=2048):

Strategy	KV Cache Size
FP16 uniform	128 MB
INT8 uniform	64 MB
CFIE per-layer	~72 MB (10% FP16 + 90% INT8)

The per-layer approach uses slightly more memory than uniform INT8 but preserves quality on sensitive layers — a better quality-memory tradeoff that uniform approaches cannot express.

2.6 Optimization 6: CSHA-Decode Fusion

Problem: During decode, each token attends to all previous tokens. The attention pattern is fundamentally different from prefill: Q has 1 row, K/V have seq_len rows. Standard FlashAttention kernels are optimized for prefill (many Q rows); decode uses a separate kernel path.

NSL's approach: CSHA Level 1 and 2 boundary fusion (from our training research) applied to the decode phase. In decode:

- **RMSNorm** is fused as a prologue to the Q/K/V projection (same as training)
- **RoPE** is fused as an epilogue to Q/K projection (same as training)
- **Single-token attention** is fused with the output projection ($Q @ K^T @ V @ W_o$ in one kernel)

The decode-specific optimization: for batch=1, the Q projection is a **matrix-vector multiply** ($W_q \times x$, where x is $[1, d_{\text{model}}]$). This is memory-bound on all GPUs. CSHA fuses it with the attention computation so the Q vector never leaves SMEM:


```

CSHA Decode Kernel:
  Load x → SMEM
  RMSNorm(x) → SMEM
  q = Wq @ x_normed → registers (matvec, not matmul)
  k = Wk @ x_normed → registers → write to KV cache
  v = Wv @ x_normed → registers → write to KV cache
  RoPE(q, k) → registers
  attn_out = FlashAttention_decode(q, kv_cache) → registers
  o = Wo @ attn_out → SMEM
  residual add → SMEM
  Store to HBM

```

Kernel launches per attention sublayer: 1 (vs 6+ in standard: norm, Wq, Wk, Wv, RoPE, attention, Wo, residual).

3. NSL Language Surface

```

nsl

serve(model=target_model, port=8080):
    max_seq = 4096
    max_batch = 64
    kv_layout = static
    kv_quant = auto                # per-layer precision from weight analysis

    speculative:
        draft = DraftModel.load("draft.nslm")
        tokens = 5
        method = tree
        tree_width = 2

    sampling:
        temperature = 0.7
        top_k = 50
        top_p = 0.9
        fused = true                # fused decode-sample kernel

    grammar:
        schema = JsonSchema("output_schema.json") # compiled DFA fused into sampling

    on_request(prompt, params):
        return generate(target_model, prompt, params)

```

```
bash
```

```
$ nsl build --serve --weights model.safetensors --target h100 server.nsl
```

```
=== CFIE Inference Build Report ===
```

```
Target: H100-SXM (80GB)
```

```
Model: NSLCoder-50M
```

```
Optimizations applied:
```

- [1] Static KV layout: 968K max tokens, 256-token blocks, direct indexing
- [2] Fused decode-sample: LM_head+softmax+topk+topp+sample → 1 kernel
- [3] Compiled speculative: tree(width=2, depth=5), draft+verify fused
- [4] Persistent decode blocks: CSHA-L2 (L3 infeasible at d=512 on H100 – SMEM ok but not needed)
- [5] Per-layer KV quant: layers 0,7=FP16, layers 1-6=INT8
- [6] Grammar DFA: 47 states, 892 transitions, fused into sampling kernel

```
Kernel launches per token: 34 (vs ~500 baseline)
```

```
Estimated decode latency: 82µs/token (vs 210µs baseline)
```

```
Estimated throughput (batch=32): 18,400 tok/s
```

4. Why PyTorch/vLLM Cannot Do This

Capability	vLLM/SGLang/TRT-LLM	CFIE
KV cache allocation	Runtime paged (block table lookup per attention)	Compile-time static (direct indexing)
Decode sampling	4-6 separate kernels, logits in HBM	1 fused kernel, logits never leave SMEM
Speculative tree	Runtime tree construction, dynamic masks	Compile-time tree in kernel control flow
Kernel launches/ token	~500-1000	~34
Grammar FSM	Runtime Python interpreter (Outlines/lm-format-enforcer)	Compiled DFA as kernel constant
KV quantization	Uniform per-model (INT8 or FP16)	Per-layer from weight spectral analysis
Attention fusion (decode)	Separate norm/proj/RoPE/attn/out kernels	CSHA-L2 fused decode kernel

5. Implementation Plan

Component	Location	LOC (est.)	Description
Static KV planner	<code>ns1-codegen/src/cfie_kv_plan.rs</code>	400	Compile-time KV layout from model+GPU specs
Fused decode-sample	<code>ns1-codegen/src/cfie_fused_sample.rs</code>	600	PTX gen for LM_head+softmax+topk+sample
Compiled speculative	<code>ns1-codegen/src/cfie_speculative.rs</code>	500	Fused draft+verify with tree structure
Persistent decode kernel	<code>ns1-codegen/src/cfie_persistent.rs</code>	700	CSHA-L2/L3 for decode phase
Per-layer KV quant	<code>ns1-codegen/src/cfie_kv_quant.rs</code>	300	Weight-spectral KV precision selection
Grammar kernel fusion	<code>ns1-codegen/src/cfie_grammar.rs</code>	400	DFA-in-kernel from grammar_compiler output
WGGO integration	<code>ns1-codegen/src/wggo.rs</code>	200	Inference decisions in WGGO Level 2 ILP

Total: ~3,100 LOC. Dependencies: flash_attention.rs, serve.rs, grammar_compiler.rs, speculative/, kv_compress/, cost_model.rs

References

1. Kwon, W., et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention." SOSP 2023.
2. Ye, Z., et al. "FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving." 2025.
3. Leviathan, Y., et al. "Fast Inference from Transformers via Speculative Decoding." ICML 2023.
4. Zheng, L., et al. "SGLang: Efficient Execution of Structured Language Model Programs." 2025.
5. NVIDIA. "TensorRT-LLM." 2024.